

Pendientes para próximas specs

Estado al 2026-09-24, rama dev. Este documento es la fuente para armar las specs que siguen. Cada bloque trae el contexto ya investigado para que nadie tenga que re-descubrirlo.

0. Qué ya está hecho (no rehacer)

| Área | Estado | --- | Backend FastAPI S1-S8 | completo · 83 tests (82 pasan + 1 pre-existente acoplado al relodj) | Django S9 (login, lista, registrar, modal ticket, sidebar, auxiliares) | completo · 35 tests | Dashboard S10 | completo (drill-down, ficha, calendario, sankey, donas, scatter, radar, presencia) | Selector de jerarquía en/sopORTE | árbol con filtro (reemplazó los 3 desplegables) | Paginación de la lista | 50 "Ver más" (conserva filtros) | Esquema reproducible | alembic upgrade head crea las 6 tablas (antes era imposible: el baseline era no-op) | Código estable (slug) | columna Código única en los 3 niveles, backfill hecho, renombrar no la toca | API de escritura de jerarquía | CRUD de los 3 niveles + propagación de FK a las atenciones PATCH /atenciones/{id}/jerarquia | Pantalla /jerarquia | árbol + detalle + alta/mover/renombrar/desactivar/eliminar | Carga de datos | 276 atenciones de septiembre (igual que prod al 22-sep), 53 áreas, 3 sectores, 4 grupos · invariante jerarquía incoherente = 0 | Backup de prod | pipeline arreglado (estaba generando 1.1M de archivos vacíos) + dump verificado por checksum | S13 (2026-09-24) | baja de usuarios (Usuarios.Activo, pantalla /usuarios/, API de alta/baja) + import histórico de los 2 técnicos retirados (556 atenciones). Espejo en docs/spec-s13-usuarios-e-import-historico.md |

Pendientes nuevos (2026-09-24)

A2 · Responsive:/atenciones en móvil se ve mal

Medido el 2026-09-24 a 390px de ancho. **Fase A hecha:** las 7 tablas van dentro de .tabla-scroll (scrollean en vez de perder columnas), .page-wide con box-sizing, y en móvil envuelven page-head, .filtros y .filtros-fila. Resultado: **10/10 pantallas en 0px de desborde** (antes: /horarios 558, /atenciones 172, /dashboard 142, / 55, /usuarios 45).

Fase B hecha (2026-09-24): /atenciones y /horarios pasan a **tarjetas apiladas** por debajo de 767px (arriba, tabla: a 768 entra cómoda). En /atenciones cada atención es una tarjeta con el botón de detalle a lo ancho. En /horarios las 3 grillas editables se transforman por CSS sin duplicar markup · duplicar habría metido dos veces el mismo <select> dentro del form y el campo viajaría dos veces · y cada control queda con su etiqueta vía data-label.

Medido: **10/10 pantallas en 0px de desborde en 320/360/390/767/768/900/1280.**

Detalle cosmético menor: los controles nativos de hora quedan ajustados en formato 12h a 390px.

O1b · Cambiar la contraseña no invalida las sesiones abiertas **HECHO** (2026-09-24)

El JWT no dependía del hash, así que una sesión vieja seguía válida tras el cambio implementado con Usuarios.TokenVersion (migración 0007): el token lleva la versión que tenía el usuario al emitirse y motivo_de_rechazo() rechaza lo que no coincide. /perfil re-loguea después del cambio para no botar la sesión propia.

Hallazgo al hacerlo: ws._autenticar no validaba NADA (ni activo): un usuario dado de baja podía abrir un WebSocket con su token viejo hasta el vencimiento (365 días). Cerrado, el chequeo quedó en un solo lugar para que HTTP y WebSocket no se desincronicen.

J2b · Catálogo: renombres y movimientos pendientes (decididos, nunca aplicados)

El **mover** de /jerarquia ya está arreglado (2026-09-24): era un solo select ambiguo condos controles que se contradecían. Ahora son dos grupos explícitos: "Directa del sector" y "Dentro de una dependencia". Los renombres de abajo siguen pendientes.

- Extras · Instituciones (sigue siendo sector: son 3, siempre)
- Eventos sale de Extras y pasa a **dependencia de Administrativos** renombrada Espacios comunes; se le suman Sala Magna y Sala de lectura
- Sala 1/2/3 (Directorio) · Instituciones · Directorio
- Sala de Docentes se queda en Vicerrectorado

· **Hacer J1 ANTES que esto:** seed_catalogo resuelve el sector por nombre desde PADRES (hardcodeado en scripts/seed.py) y mapeo-areas-dedup.csv guarda grupo_padre por nombre. Renombrar un sector hoy los rompe.

J1 · Bootstrap por código **HECHO** (2026-09-25)

Verificado el 2026-09-25: base fresca + alembic upgrade head + seed_catalogo = catálogo idéntico (3+4+56). Y el round-trip: renombrar Biblioteca Central en la BD, exportar_catalogo, vaciar, re-seed · el renombre persiste. Lo de abajo es el contexto original (ya cumplido).

Por qué: la pantalla /jerarquia ya cambia el catálogo, pero el bootstrap(seed.py) lee CSV **por nombre**. Un renombre deja los CSV viejos y un re-seed sobre una BD vacía reconstruye la estructura anterior, en silencio.

Por qué no se hizo con un export simple el export tendría que reescribir la columna area del CSV de atenciones emparejando filas con la BD, y **hay descripciones duplicadas** (5 grupos con misma descripción y fecha) · asignaría áreas equivocadas sin avisar. Se descartó a propósito.

La solución que el slug ya habilita que los CSV referencien por **código**

| Archivo | Cambio | |---|---| | mapeo-areas.csv | nuevo_nombre · codigo_destino (144 filas, conversión única) | atenciones_septiembre.csv | columna area · area_codigo | | backend-fastapi/scripts/extraer_septiembre.py | resolver el texto de prod · **código** (vía mapeo + catálogo) | | backend-fastapi/scripts/seed.py | resolver el área por **código** | | backend-fastapi/scripts/exportar_catalogo.py | **nuevo:** escribe el catálogo (con códigos) desde la BD. No toca el CSV de atenciones |

DoD: sobre una BD vacía alembic upgrade head + seed.py reconstruye la estructura **actual** (incluidos renombres y movimientos hechos por la GUI).

E1 · Especialidades / responsabilidades (spec futura)

Requisito del usuario: la "especialidad" es un texto libre y no alcanza. Los dos jefes tienen **roles distintos**, y entre los técnicos hay encargados de **accesos biométricos**, otros de **videovigilancia**, otros de **redes**, etc. Hay que modelarlo en serio.

Preguntas para la spec (no resueltas):

- ¿Una sola especialidad por persona **varias?** (parece que varias: alguien puede estar en redes y en videovigilancia)
 - ¿Es lo mismo que área de responsabilidad o son dos cosas? (la jerarquía ya modela el lugar; la especialidad es el saber hacer)
-

- ¿Sirve para **enrutar** el trabajo (asignar a quien sabe) o solo para mostrarlo?
- ¿Los jefes necesitan algo distinto, dado que coordinan en vez de ejecutar?
- ¿Hace falta historial (quién sabía qué y desde cuándo)?

Hallazgo relacionado: Usuarios. Especialidad es Text libre y ya se muestra en el listado del equipo (GET /api/usuarios). PATCH /api/usuarios/{id}/especialidad existe y es la base sobre la que construir.

N2 · Bloc de notas flotante (widget)

Requisito del usuario: además de la pantalla propia, el bloc de notas debería poder ser un **widget flotante**, editable y de **tamaño ajustable**, disponible desde cualquier pantalla (hoy el sidebar tiene la entrada, que lleva a /notas).

Lo que hay que decidir: cómo se abre (botón del sidebar, atajo de teclado), flota sobre el contenido o se ancla a un costado, si recuerda posición y tamaño (otra preferencia de presentación · localStorage), y cómo convive con el autoguardado que ya tiene.

A1 · Deuda de accesibilidad

- **La CSS está 100% en px** (271 usos de px, 0 de rem). Por eso el tamaño de letra simplemente con zoom, que escala todo proporcionalmente. Funciona, pero el control fino (escalar solo el texto, no los espacios) requiere migrar a rem. **Síntoma medido el 2026-09-24** con la letra en grande o x1 persistida, las pantallas desbordan a anchos intermedios (/atenciones 159px, /dashboard 134, /horarios 108 a 910px con x1). Causa: zoom no afecta a las media queries, así que a 910px entra el layout de escritorio (el corte es a 900) pero todo mide 1,3× y no cabe. Migrar a rem lo resuelve de raíz.
 - **Pendiente una pasada de accesibilidad** por las pantallas existentes (foco, teclado, contraste, aria), no solo por las preferencias del perfil. Hay un skill **accessibility** disponible.
-

J2 · Jerarquía, lo que quedó fuera de la v1

- **El catálogo sigue con "Extras"** y sin los movimientos y renombres ya decididos ("Espacios comunes", subir Eventos, mover Sala Magna y Sala de lectura, las Salas 1/2/3 a Directorio). Verificado el 2026-09-23: **nunca se aplicó**, ni en el catálogo ni en el CSV. Lo hace el usuario desde /jerarquia.
 - **Movimiento en lote** (seleccionar varias áreas y moverlas juntas). Decisión del usuario: lo hace a mano. Solo tiene sentido si agrupar 24 áreas se vuelve tedioso.
 - **Desactivar sectores:** GruposPadres **no tiene columna Activo** (solo Nombre, Descripcion, Orden). Hoy un sector solo se renombra, reordena, o se borra si está vacío. Agregar la columna es aditivo.
 - **Sin drag & drop, a propósito** mover reescribe N atenciones; un arrastre accidental sería un error silencioso. Se usa un diálogo explícito con el aviso de impacto. No cambiarlo sin decidirlo.
 - **Unicidad de las áreas sin dependencia** el UniqueConstraint (GrupoPadreId, GrupoId, Nombre) **no protege** a las 36 áreas con GrupoId NULL (Postgres trata cada NULL como distinto). La unicidad hoy la garantiza el chequeo de la app.
-

P1 · /perfil · personalización y accesibilidad

Requisito del usuario: cada uno debe poder ver **su propio** perfil y jugar con sus estadísticas. Los **Jefes** entran a `/perfil` y ven el suyo y el de los técnicos. Ahí se retrabajan los 3 gráficos de técnicos que hoy viven en el dashboard (scatter "carga vs fuera de turno", "rendimiento por técnico", "colaboraciones") más el radar, que se aplana cuando el filtro tiene una sola categoría dominante.

Hallazgo que define el trabajo: GET `/api/atenciones/stats` ya acepta `usuario_id` · no hay que agregar agregaciones. **Pero exige** `is_privileged`, así que un técnico `can_view_dashboard` **no puede pedir ni sus propias stats**. Decisión necesaria: endpoint propio (`/api/atenciones/mis-stats`, solo uno mismo) o permitir el self en `get_stats`.

Permisos: técnico · solo lo suyo. Jefe · lo suyo y el de cualquier técnico.

Etapas
Etapas 1 · HECHO perfil con identidad, especialidad editable, cambio de contraseña propio POST `/api/auth/password`, mínimo 8 caracteres) y preferencias de accesibilidad (tamaño de letra, contraste, movimiento) guardadas en **localStorage** · son de presentación, no viajan con la cuenta · El layout ya deja reservado el lugar de la etapa 2.

Etapas 2 · PENDIENTE Las estadísticas por técnico (los 3 gráficos + radar).

Accesibilidad: es transversal, no solo de `/perfil`. Hay un skill `accessibility` disponible. Vale una pasada por todas las pantallas (foco, teclado, contraste, aria), no solo por el perfil.

N1 · /notas · bloc de notas · HECHO

Verificado el 2026-09-24: la ruta, la vista y el template existen. Lo de abajo es el contexto original.

El más barato de todos **el backend ya está listo** (GET/PUT `/api/usuarios/notas`, un texto por usuario en `Usuarios.Notas`). Falta solo la pantalla: un textarea con guardado. Hoy el sidebar tiene el botón "Bloc de notas" sin acción.

H1 · /horarios · HECHO

Verificado el 2026-09-24: los 4 cambios de backend están aplicados (`DiaSemana` en el unique, `ROLES_CON_HORARIO`, cobertura separada laborable/sábado, `label` generado desde las horas), la pantalla existe y hay 52 horarios cargados. · efecto colateral" que advertía esta sección NO aplica. Lo de abajo es el contexto original.

Backend **casi completo**: GET/POST (upsert por técnico+mes) DELETE `/api/horarios` + GET `/api/horarios/cobertura` (ya calcula, para los 4 bloques fijos del día, quién cubre cada uno). Wireframe hecho: docs/wireframes/horarios.excalidraw.

Cómo funciona el equipo (datos del usuario)

- De **lunes a viernes** el equipo cubre **08:00-20:00**; cada técnico hace su turno escalonado (07-15, 08-16, 09-17, 10:30-18:30, 12-20).
 - **Sábado es distinto** solo 2 turnos · mañana **08:00-12:00** (5 técnicos) y tarde **14:30-18:30** (2 técnicos). Los 2 de la tarde **rotan cada mes** y la rotación se cambia **a mano**.
 - **Domingo no trabaja nadie** · todo el día fuera de turno.
 - Los **jefes** (Wilmer Cerruto y Josue Huayllas) tienen turno **fijo 08:00-12:00 + 14:30-18:30** todos los meses y **no cuentan para la cobertura**: su horario existe solo para saber si están dentro o fuera de turno.
 - El mediodía y la noche del sábado **no los cubre nadie, y es esperado**.
 - Los 4 bloques de cobertura (08-12, 12-14:30, 14:30-18:30, 18:30-20) son **fijos y son la realidad**: no hace falta hacerlos configurables ni extenderlos.
-

Los 4 cambios de backend que implica (en el wireframe):

- 1 **Falta el día en el modelo** Horarios es uno por (técnico, mes) y asume el mismo turno todos los días. Con el sábado distinto, la clave única pasa a incluir día. El guardado **es por día** (L-V un bloque, sábado otro), aunque la UI muestra solo dos bloques.
- 2 **Los jefes tienen que poder tener horario:** hoy POST /api/horarios rechaza todo lo que no sea role == "Tecnico".
- 3 **Un día sin turno = fuera de turno** Hoy, sin horario, estado_efectivo() asume "sin restricción" y muestra Disponible; así el domingo no funcionaría.
- 4 **La cobertura se separa en dos** (lunes a viernes / sábado): no pueden compartirla.

Efecto colateral: mientras septiembre esté vacío, los 7 técnicos y los 2 jefes van a figurar **fuera de turno todos los días**. Es correcto y se arregla cargando el mes (el usuario ya dijo que lo hará).

Otras inconsistencias detectadas:

- GET /api/horarios usa role == "Jefe" en vez de is_privileged(), que es lo que usa el resto de la app: si quien administra no tiene role == "Jefe", la pantalla le muestra solo su fila.
- El label es texto libre redundante con las horas: conviene generarlo siempre desde las horas (v2 ya lo hacía así).

Lo que ya está resuelto las 5 plantillas de turno de v2 (08:00-16:00, 08:00-12:00 + 14:30-18:30, 12:00-20:00, 07:00-15:00, 09:00-17:00), el guardado por fila y global, y "copiar el mes anterior" como ayuda para no cargar 7 técnicos amano cada mes.

R1 ./reportes **HECHO** (2026-09-25)

Confirmado que sí era un feature real el jefe (Wilmer) presenta mes a mes el trabajo del equipo al rectorado. Spec: docs/spec-s14-reportes.md. Lo de abajo es el contexto original.

Criterios que fijó el usuario solo imprimible (sin export a Excel) **sin sección por técnico** (el rectorado no necesita eso) **mes elegible + acumulado del año**

Implementado: /reportes/?mes=YYYY-MM&vista=mes|anio con 5 KPIs comparados al mes anterior, evolución del año, categorías, sectores, medios y tipo de solicitante, top áreas, 3 casos destacados y nota de metodología. Imprime a PDF con @media print (A4, oculta nav y selector, break-inside: avoid) sin librerías nuevas.

Decisión abierta (D-01) los "destacados" salen de una heurística (la solución más larga cada categoría top-3). La alternativa es un flag manual **destacar** en la atención, que requiere backend.

L1 ./launcher **hecho** (pantalla "Inicio")

Quedó como pantalla de entrada (/) para técnicos y jefes; el auxiliar sigue yendo a /auxiliares/. Trae: anuncio del equipo (**uno solo**, con autor y hora, sin persistir - se borra al reiniciar); El equipo ahora" con los chips y las tarjetas del dashboard más el turno del día y la carga, accesos rápidos **gateados con can_dashboard** (Dashboard y Jerarquía solo para jefes), y el toggle de estado, que queda deshabilitado y **rechazado por el backend** si estás fuera de turno.

Referencia en v2: frontend/src/app/launcher/page.tsx (112 líneas). No hizo falta backend nuevo: se enriqueció /api/usuarios (horario_hoy, entra_a_las, atenciones_hoy, puede_cambiar_estado) y /api/announcements (autor y hora).

O1 · Operación y seguridad

- **HTTPS:** sigue en HTTP plano (decidido: se accede por LAN y por Tailscale, y el tráfico de Tailscale ya va cifrado por WireGuard). Por la LAN pelada la contraseña viaja sin cifrar: es el riesgo aceptado hoy. La config está lista detrás de un flag `DJANGO_HTTPS=1` + un proxy que mande `X-Forwarded-Proto: https` activa cookies seguras, redirección a HTTPS y HSTS. Queda apagado por defecto para no romper el acceso actual. (Si el proxy no manda ese header, es bucle infinito.)
- **Cambiar la contraseña no cierra las sesiones abiertas** Descubierto el 2026-09-23: una sesión iniciada antes del cambio sigue válida. Es de bajo riesgo (la propia persona), pero lo correcto es invalidarlas al cambiar la contraseña. Falta hacerlo.
- **Runbook:** `runbook-operacion.md` como correr los procesos, las variables de entorno, la carga de datos, los horarios, los backups y el checklist previo a exponerlo a los usuarios.
- **Backups:** el pipeline quedó arreglado, pero no hay alerta si vuelve a fallar. Vale un chequeo (tamaño > 0 y antigüedad < 2 días).
- **Cutover (S11) · ejecutado el 2026-09-23** La v2 es producción desde esa fecha arranca con 276 atenciones (septiembre = mes 1), el catálogo y los horarios de septiembre. Los 9 usuarios entran con su contraseña de v1 (se copiaron los hashes bcrypt); el auxiliar quedó sin usar y con contraseña aleatoria.
 - La v1 quedó **apagada pero entera** (`docker start soporte-backend` `soporte-frontend` la revive en `:3002`) y con dump final en `~/backups/soporte/prod_v1_final_*.dump`.
 - `DJANGO_DEBUG=0 + runserver --insecure` (los estáticos con Debug apagado).
 - Las unidades de `deploy/systemd/` quedaron **instaladas y habilitadas** el 2026-09-23: arrancan al boot y se reinician solas (probado con SIGKILL).
 - La base de producción vive en el contenedor `soporte-postgres-dev (:5433)`, que es un nombre engañoso heredado: el contenedor `soporte-postgres` es la base vieja de v1.

D1 · Datos

- **La historia es solo septiembre (276 de 2214).** Decisión del usuario: v2 arranca de 0 y v1 queda como evidencia de trabajo. Consecuencia: **S12 (migración de historia) sale del roadmap.**
- **atenciones_septiembre.csv es un snapshot:** prod sigue recibiendo atenciones (253 · 274 · 276, y sigue). El flujo para ponerse al día son dos comandos desde `backend-fastapi/`:

```
python scripts/extraer_septiembre.py      # refresca el CSV desde prod
python scripts/actualizar_atenciones.py    # suma a la base solo lo que falta
```

`seed_atenciones` quedó idempotente por `created_at`, así que ambos se pueden correr las veces que haga falta. No hace falta el seed completo, que además se hashea las contraseñas de todos.
- **56 filas con duplicados** (fecha + área + categoría + descripción, grupos de 2 y 3). Puede ser legítimo (varias personas pidiendo lo mismo el mismo día). Sin revisar.
- **2 áreas huérfanas** (atenciones): Sala 2 (Directorio) y Sala 3 (Directorio).
- **Los 25 registros sin jerarquía del dev viejo quedaron obsoletos** se borraron al cargar de 0, y la carga nueva tiene 0 sin área. Ya no hay nada que sanear.

Decisiones ya tomadas (no volver a preguntar)

- Sectores: **siempre 3** · Administrativos · Académicos · Instituciones (Extras se renombra a Instituciones). Es a propósito: no hay un 4to sector ni columna Activo en GruposPadres, para que no puedan crearse más de 3.
 - Eventos sale de Extras y pasa a **dependencia de Administrativos** renombrada "**Espacios comunes**"; se le suman Sala Magna y Sala de lectura. Sala de Docentes **se queda** en Vicerrectorado.
 - Sala 1/2/3 (Directorio) pertenecen a Instituciones · Directorio
 - El sector de terceros se llama "**Instituciones**" (el jefe rechazó "Externos" por sonar excluyente). EIAE es posgrado y es empresa aparte · va como dependencia.
 - Las empresas con subdivisiones van a nivel **dependencia**; sin subdivisiones, quedan como área directa del sector.
 - Nombres de los niveles en la UI: **Sector · Dependencia · Área**
 - "Batch" · "**Lista**".
 - Movimiento en lote: **descartado** (el usuario lo hace a mano).
-

Hallazgos que no hay que volver a investigar

- **No hay tablas fantasma** en la BD: 6 de negocio + __EFMigrationsHistory + alembic_version.
 - **Categorías limpias** las 8 de la BD son exactamente las 8 válidas.
 - alembic/script.py.mako **faltaba** alembic revision estaba roto en todo el proyecto. Agregado.
 - 0001_baseline **es no-op intencional** (el esquema lo poseía EF). La migración es de 1d87cc0b82.
 - seed.py **siembra 10 usuarios**, no 9 (el docstring decía 9).
 - **El PUT de atenciones** dejaba los 2 FK desincronizados si el cliente mandaba solo area_id. Arreglado.
 - SessionLocal es autoflush=False y los modelos **no declaran** relationship(): hay que forzar los flush() en orden al sembrar.
 - **Prod no tiene puerto mapeado al host**: se accede con docker exec soporte-postgres psql -U soporte -d soporte. IPs reales: 10.0.13.3 (prod) y 10.0.18.3 (dev), no .2.
 - docs/traspaso-datos-septiembre.md (de otro agente) quedó desactualizado: §4 dice 45 con colaborador (ahora son 55) y §7 recién ahora es cierto.
-